Relazione 1: titolo da inserire

Nome e cognome

18 ottobre 2026

Indice

1 Introduzione **2**

2 Primo punto: Phishing e AUR **2**

2.1 Il phishing e la compromissione dell'account 2

2.2 Dalla pubblicazione malevola all'esecuzione nel browser 3

3 Ricostruzione degli episodi **4**

3.1 Cronologia essenziale e componenti coinvolti 4

3.2 Risposta adottata 4

4 Conclusioni **4**

Bibliografia **4**

1 Introduzione

Hello World!

2 Primo punto: Phishing e AUR

2.1 Il phishing e la compromissione dell'account

La compromissione dell'account è iniziata con un'e-mail di phishing proveniente dall'indirizzo support@npmjs.help, associato a un dominio registrato appena tre giorni prima dell'attacco e un server localizzato nei Paesi Bassi.

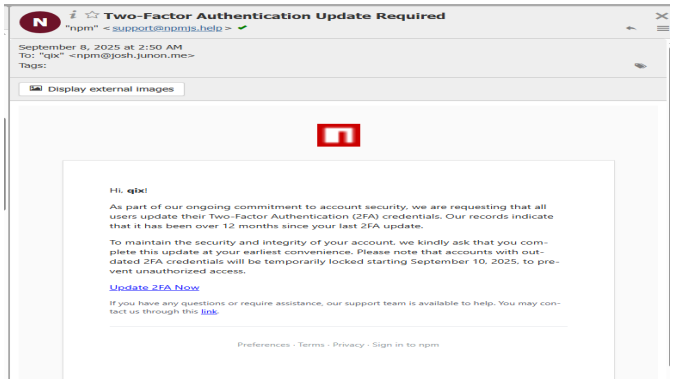


Figura 1: Mail di phishing ricevuta dall'utente.
Fonte: [1]

Il messaggio minacciava la sospensione dell'account se entro due giorni non fosse stato effettuato il login, un chiaro tentativo di creare urgenza e pressione psicologica sull'utente. Qualche ora dopo, più precisamente alle 13:16 UTC, i sistemi di Aikido, un'azienda specializzata in cybersecurity, hanno notato che l'account di Josh stava pubblicando pacchetti con codice apparentemente malevolo, permettendo in seguito di capire che era stato compromesso.

Josh ha dichiarato che, a causa di una settimana stressante e della credibilità dell'e-mail, aveva cliccato sul link mentre utilizzava lo smartphone, anziché accedere direttamente al sito ufficiale di npm come era solito fare. Dopo aver inserito le proprie credenziali, il suo account è stato compromesso impossibilitandone l'accesso. Tutti i pacchetti compromessi contenevano codice JavaScript che intercettava richieste e risposte di rete e interazioni con i wallet. Quando veniva trovato un indirizzo di un wallet, il codice lo sostituiva con un indirizzo controllato dall'attaccante per poter reindirizzare i fondi verso il proprio. Per complicare la comprensione del codice malevolo, l'attaccante ha utilizzato tecniche di offuscamento del codice come l'uso di variabili con codici, come per esempio la variabile 0x112fa8. [1]

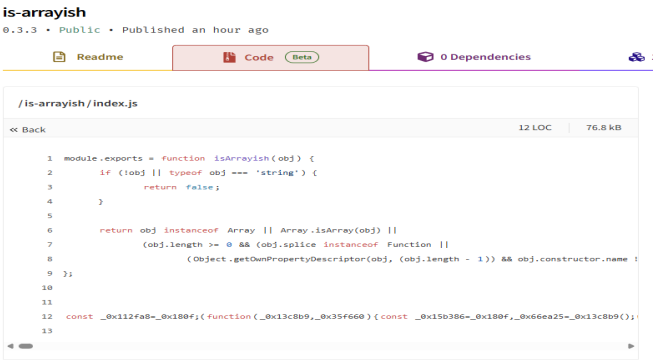


Figura 2: estratto del codice malevolo presente nei pacchetti compromessi
Fonte: [1]

Per portare a termine l'attacco non era necessario compromettere l'intero registro npm, è stato infatti sufficiente compromettere l'account di un publisher e sfruttarne i permessi per distribuire versioni malevole dei pacchetti per raggiungere l'obiettivo desiderato.

2.2 Dalla pubblicazione malevola all'esecuzione nel browser

L'attaccante, a questo punto, ha pubblicato diverse versioni malevole dei pacchetti gestiti dal manutentore. Tra queste era presente debug@4.4.2 che manteneva le stesse funzionalità della versione originale ma contenente il codice malevolo. L'attacco aveva un'ampia superficie di esposizione: secondo Aikido, i 18 pacchetti inizialmente compromessi avevano oltre 2 miliardi di download settimanali. Il codice poteva raggiungere le applicazioni attraverso le dipendenze delle librerie. Infatti, quando uno sviluppatore installava o aggiornava un pacchetto nel suo progetto, poteva anche scaricare una versione compromessa come debug@4.4.2 sia direttamente che come dipendenza di un'altra libreria. Durante la fase di build, eseguita localmente oppure attraverso processi automatizzati di integrazione e distribuzione continua (CI/CD) come per esempio il build automatico a seguito di un push, il codice malevolo poteva essere incorporato nel bundle, ovvero l'insieme di file contenenti il codice e le dipendenze necessarie all'applicazione. Una volta distribuita l'applicazione, il codice poteva essere eseguito nei browser degli utenti finali, interferendo con eventuali operazioni relative ai wallet di criptovalute. Secondo l'advisory ufficiale di debug presente su GitHub, il comportamento malevolo documentato riguardava esclusivamente gli ambienti browser, mentre le applicazioni eseguite su server o da riga di comando (CLI) non erano interessate da questo specifico attacco. [1] [2]

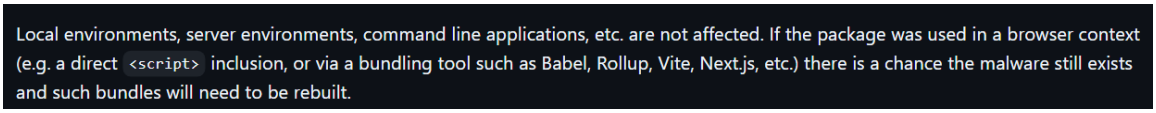


Figura 3: Chiarimento sugli ambienti interessati dall'attacco
Fonte: [3]

3 Ricostruzione degli episodi

3.1 Cronologia essenziale e componenti coinvolti

L'8 settembre 2025 l'account npm del manutentore di pacchetti Josh, è stato compromesso a seguito di un attacco di phishing, portando alla pubblicazione di versioni malevole di pacchetti. Alle 13:16 UTC, i sistemi di Aikido Security hanno rilevato la pubblicazione di diversi pacchetti npm contenenti codice sospetto, l'azienda ha successivamente avvisato il manutentore.

Poche ore dopo lo stesso utente ha dichiarato di aver perso il controllo dell'account e di essere al lavoro per risolvere il problema. Secondo Aikido il manutentore sembrava essere riuscito a rimuovere parte dei pacchetti compromessi prima di perdere l'accesso al proprio account. Nello stesso giorno inoltre, furono aperte anche alcune issue sui repository GitHub, come la issue 656 per chalk 5.6.1 e la 1005 per debug 4.4.2, per segnalare la presenza di codice malevolo nelle versioni appena pubblicate su npm e per avvertire gli sviluppatori. [1] [4]

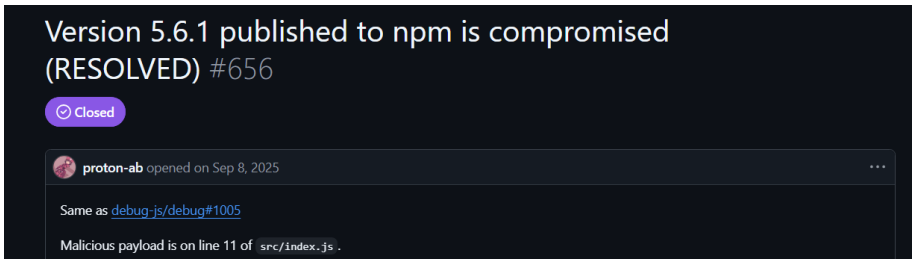


Figura 4: Issue GitHub relativa alla compromissione del pacchetto chalk@5.6.1.
Fonte: [4]

3.2 Risposta adottata

Durante la giornata dell'8 Settembre npm ha rimosso le versioni compromesse dei pacchetti per evitare ulteriori download. La patch correttiva per debug arriva solamente il 13 Settembre, quando il manutentore rilascia la versione sicura 4.4.3 accompagnata da alcune indicazioni per gli sviluppatori: aggiornare i pacchetti alle versioni sicure, eliminare completamente la cartella `node_modules`, svuotare la cache del gestore dei pacchetti e ricostruire da zero i bundle creati con i pacchetti compromessi. [3]

4 Conclusioni

Bibliografia

[1] Aikido Security. *npm debug and chalk packages compromised.* 2025. URL: `https://www.aikido.dev/blog/npm-debug-and-chalk-packages-compromised` (visitato il giorno 18/10/2026).

[2] Wiz.io. *Widespread NPM Supply Chain Attack: Breaking Down the Impact and Scope Across debug and chalk*. 2025. URL: https://www.wiz.io/blog/widespread-npm-supply-chain-attack-breaking-down-impact-scope-across-debug-chalk (visitato il giorno 18/10/2026).

[3] debug maintainers. *debug@4.4.2 contains malware after npm account takeover*. 2025. URL: https://github.com/debug-js/debug/security/advisories/GHSA-4x49-vf9v-38px (visitato il giorno 18/10/2026).

[4] Chalk contributors. *Issue 656: chalk@5.6.1 compromised*. 2025. URL: https://github.com/chalk/chalk/issues/656 (visitato il giorno 18/10/2026).